

Abteilung Informatik

Modul: WE1 HS21

Prüfung 21.1.2021, 08:50-10:50

MIT LÖSUNG**Name:** _____**Zeit:** 120 Minuten**Hilfsmittel:** 1 Seite A4 (ein- oder doppelseitig beschrieben)

1. Legen Sie Ihre Legitimationskarte auf das Pult.
2. Mobiltelefone ausschalten!
3. Tragen Sie Ihren Namen jetzt gleich auf diesem Deckblatt ein.
4. Kontrollieren Sie, ob Sie alle Aufgabenblätter (Seite 1 bis 20) erhalten haben.
5. Streichen Sie ungültige Lösungsteile deutlich durch. Mehrfachlösungen sind falsch.
6. Zusatzblätter erhalten Sie beim Bedarf vom Prüfer.
7. Geben Sie am Prüfungsende diese Lösung zusammen mit eventuellen Zusatzblättern ab.
8. Vorzeitige Abgabe bis maximal 15 Minuten vor Ende der Prüfung.
9. Bleiben Sie am Prüfungsende ruhig sitzen, bis alle Prüfungen eingesammelt wurden.
10. Der Weissraum lässt keine Rückschlüsse auf die Länge der erwarteten Antwort zu.
11. Trennen Sie möglichst nicht die Heftung der Prüfung.
12. Beschriften Sie lose Blätter mit Ihrem Namen.
13. Schreiben Sie Ihre Lösung nicht mit Bleistift. Verwenden Sie auch keinen roten Kugelschreiber.

Eine erfolgreiche Prüfung wünschen
Frieder Loch und Michael Gfeller

#	Aufgabe	Punkte	Erreicht
1	Usability	8	
2	HTML	17	
3	CSS	13	
4	JavaScript	20	
5	DOM	6	
6	AJAX/Rest	26	
	Total	90	

1 Usability (8P)

a) In der Vorlesung wurden Usability-Heuristiken besprochen. Eine Heuristik verlangt die "Sichtbarkeit des Systemstatus". Was besagt diese Heuristik? Erläutern Sie diese Heuristik an einem konkreten Beispiel. (2P)

Das System muss dem User jederzeit kommunizieren in welchem Zustand es ist und welche Möglichkeiten der Interaktion bestehen. Dies ist, zum Beispiel, wichtig wenn eine Operation vom System lange dauert und der Nutzer informiert werden soll, dass die Bearbeitung der Anfrage noch dauert. Ziel ist es, zu vermeiden, dass User, zum Beispiel, eine Aktion mehrfach ausführen. (1P)

Beispiel: Progress-Bar, Feedback (z.B: Deaktivieren von Buttons) (1P)

b) Die Umsetzung dieser Heuristik kann in der Praxis anspruchsvoll sein. Dies ist zum Beispiel beim Umgang mit API-Services, die unterschiedlich schnell oder unter Umständen gar nicht antworten, der Fall. Beschreiben Sie textuell (kein Programmcode), wie Sie diese Heuristik in diesem Fall umsetzen könnten. (2P)

Es ist oft nicht klar, ob und wann eine Antwort vom Server kommt. User weiss nicht, wie lange es dauert und würde unter Umständen wiederholt auf einen Button klicken und unnötige Anfragen generieren. Es muss an den Nutzer kommuniziert werden, dass die Antwort auf dem Weg ist.

Möglichkeiten zur Umsetzung: Suspend State (Buttons ausgrauen), Fortschrittsanzeige, Anfrage ggf. mehrfach senden

c) Erklären Sie den Begriff der Affordance an der folgenden Abbildung eines Türgriffs. Ist dies eine gute Affordance? Begründen Sie Ihre Antwort. Was können Sie aus diesem Beispiel für die Verwendung von Affordances bei der Gestaltung von graphischen User Interfaces ableiten? (4P)



<https://qph.fs.quoracdn.net/main-qimg-889f0b3f1eba9368b757df6068bef57f-lq>

Affordance: Wahrnehmbare Nutzungsmöglichkeit eines Gerätes. (1P für Definition)

Der Türgriff würde eher dazu motivieren, daran zu ziehen, anstatt zu drücken. Er ist somit eine eher schlechte Affordance, was auch durch die zusätzlich nötige Beschriftung angedeutet wird. (2P für plausible Diskussion)

UI-Design: z.B. Schalter und Slider korrekt verwenden (1P für Beispiel)

2 HTML (17P)

a) `<article>`, `<aside>` und `<nav>` sind Beispiele semantischer HTML-Elemente. Wieso sollten Sie semantische Elemente, anstatt von `<div>`- oder `<span>`-Elementen verwenden? Geben Sie **ein** konkretes Beispiel einer Anwendung, die von der Verwendung semantischen Markups profitiert. (3P)

Mit semantischem Markup können wir die Bedeutung des Inhalts genauer darstellen als mit `<div>/<span>`. (1P)

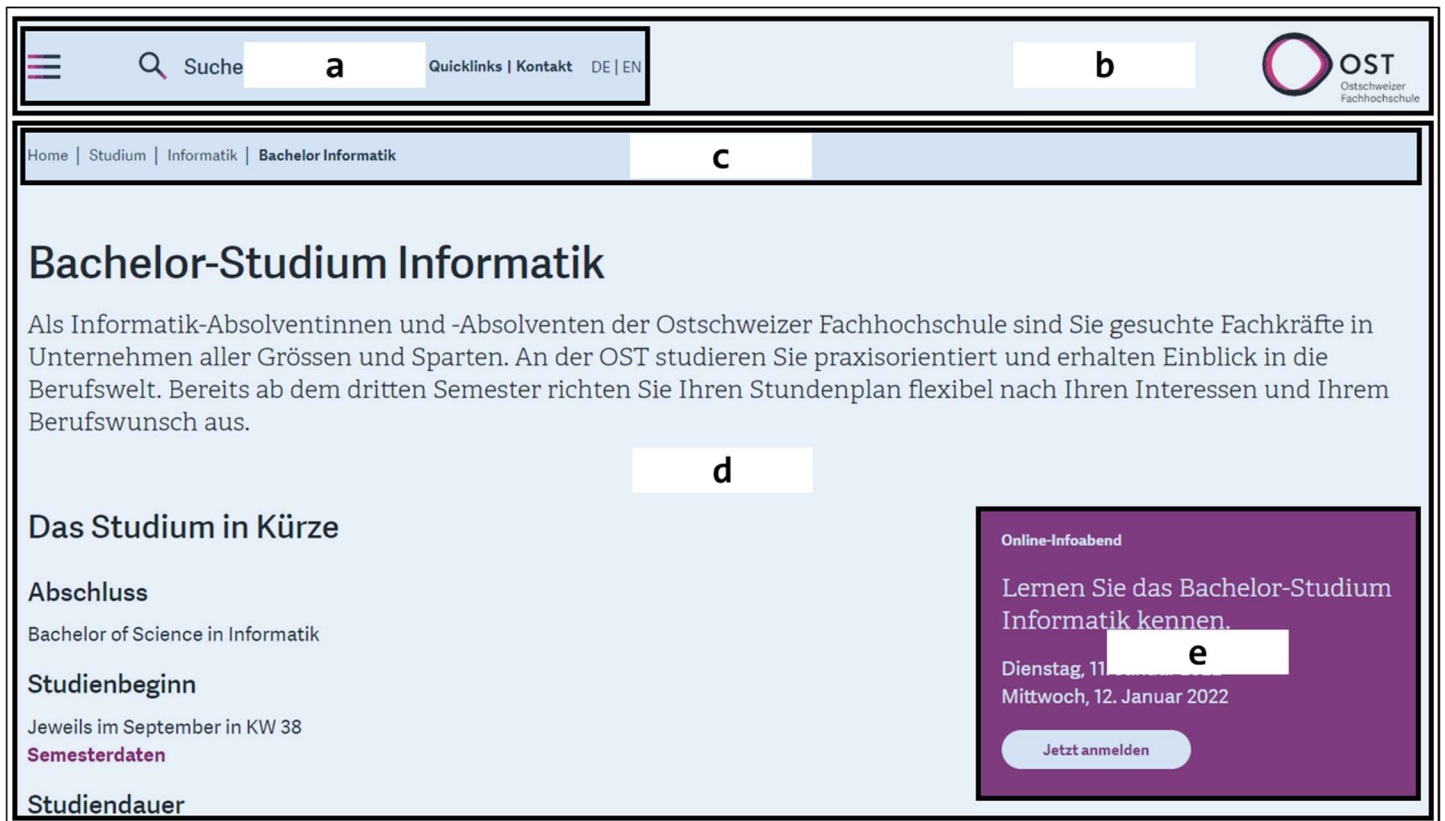
Semantic Markup ermöglicht es Crawlern oder Screenreadern den Inhalt der Seite besser zu verstehen.

Mit einem `<nav>`-Tag kann zum Beispiel ein Screen Reader erkennen, dass es sich um Navigationselemente handelt und diese entsprechend hervorheben. Bei einem `<div>` wäre dies nicht möglich (Anwendung bis 2P)

b) Weshalb sollten Sie Elemente wie `<center>`, `<font>` und `<strike>` nicht verwenden? (1P)

Layout soll mit CSS realisiert werden, deshalb sind diese Elemente deprecated.

c) Schlagen Sie für die folgende Webseite eine HTML-Struktur mit semantischem Markup vor. Wählen Sie für jeden der hervorgehobenen Bereiche ein passendes Element. Begründen Sie Ihre Auswahl kurz. (5P)



The screenshot shows the website for the Bachelor-Studium Informatik at OST. The layout includes a top navigation bar with a search bar (labeled 'a'), quicklinks, and the OST logo (labeled 'b'). Below this is a secondary navigation bar with a breadcrumb trail (labeled 'c'). The main content area (labeled 'd') features a large heading, a descriptive paragraph, and a list of study details. A sidebar on the right (labeled 'e') contains an 'Online-Infoabend' announcement with a date and a button.

a)

nav: Stellt Möglichkeiten der Navigation bereit (0,5P Wahl, 0,5P Begründung)

b)

header: Stellt globale Infos bereit (z.B. Logo, nav,...) (0,5P Wahl, 0,5P Begründung)

c)

nav: siehe oben (0,5P Wahl, 0,5P Begründung)

d)

main: Wichtigster Inhalt im Body-Element (0,5P Wahl, 0,5P Begründung)

e)

aside: Indirekt auf main-Inhalt bezogen (0,5P Wahl, 0,5P Begründung)

d) Im Anhang (Kapitel 7) finden Sie Ausschnitte aus dem HTML-Standard für die Elemente `<ul>`, `<ol>` und `<dl>`. Entscheiden Sie für die folgenden Anwendungen welche Art von Liste (`<ol>`, `<ul>`, `<dl>`) die beste Wahl ist. Begründen Sie Ihre Entscheidungen kurz. (4P)

Eine Liste von Arbeitsschritten für ein Kochrezept?

`<ol>`: Ordnung der `<li>` für Bedeutung des Dokuments relevant.

Eine Liste von Fragen und Antworten (z.B. FAQs)?

`<dl>`: Assoziationsliste mit Gruppierung von Key-Value Paaren.

Eine Liste der Spieler:innen eines Computerspiels geordnet nach Punktzahl?

`<ol>`: Ordnung der `<li>` für Bedeutung des Dokuments relevant.

Eine Liste von Büchern die Sie besitzen?

`<ul>`: Ordnung der `<li>` für Bedeutung nicht relevant. Blosser Aufzählungsliste.

Die Dokumentation des `<ol>`-Elements beinhaltet noch weitere Informationen zum Element (siehe Anhang, Kapitel 7). (4P)

e) Welche Elemente können in einem `<ol>`-Element auftreten? (2P)

`<ol>`-Elemente dürfen ausschliesslich `<li>`-Elemente beinhalten. Optional: script-supporting Elements (`<script>`/`<template>`)

f) Was ist Tag-Omission und wann kann ein `<li>`-Tag weggelassen werden? (2P)

Tag Omission: Schliessendes Tag eines Elements kann unter bestimmten Bedingungen weggelassen werden.

li: Endtag kann weggelassen werden, wenn direkt ein weiteres li-Element folgt.

3 CSS (13P)

a) Erklären Sie den Unterschied zwischen den Selektoren "section > img" und "section img". Geben Sie ein Beispiel das den Unterschied zeigt. (3P)

section > img: selektiert nur imgs die direkten Kinder eines section-Elements sind.
section img selektiert alle imgs die Nachfahren von section sind. Im folgenden Beispiel wird das img nur von der zweiten Regel selektiert. (Jeweils 1P für Erklärung der Selektoren)

```
<section>  
  my title  
  <div>  
    <img>with a second row</img>  
  </div>  
</section> (1P für Unterschied)
```

b) Folgendes HTML und CSS sind gegeben.

HTML

```
<div id=style-box class="style-box"></div>
```

CSS

```
#style-box {  
  background-color: green;  
}  
.style-box {  
  background-color: blue;  
}
```

Welche Hintergrundfarbe bekommt das div-Element? Erklären Sie das Ergebnis, indem Sie das Prinzip der Spezifität erläutern. (3P)

Grün: ID-Selektor ist spezifischer. (1P für korrektes Ergebnis)
Prinzip der Spezifität: Auflösung von Konflikten zwischen Regeln, die selbes Element betreffen (1P)
Algorithmus für Spezifität: Drei Zähler: a (z.B. ID-Selektoren) b (z.B. Klassen-Selektoren) c (z.B. Typ-Selektoren) (1P)

c) Folgendes HTML und CSS sind gegeben.

HTML

```
<div id=style-box class="style-box"></div>
```

CSS

```
#style-box {  
    background-color: green;  
}
```

```
div.style-box {  
    background-color: blue;  
}
```

Welche Hintergrundfarbe bekommt das div-Element? Erklären Sie auch hier das Ergebnis. (2P)

Immer noch **Grün** (1P)
Type-selector hat keinen Einfluss auf Spezifität. Spezifischere Regel (mit ID-Selektor) gewinnt weiterhin. (1P)

d) Folgendes HTML und CSS sind gegeben.

HTML

```
<div id=box1>Box 1</div>  
<div id=box2>Box 2</div>
```

CSS

```
#box1 {  
    border: 1px black solid;  
}
```

```
#box2 {  
    border: 1px black solid;  
}
```

Skizzieren Sie, wie das gegebene HTML auf dem Bildschirm dargestellt wird. Aus Ihrer Skizze sollten unter anderem die relative Höhe und Breite der Elemente zu einander und zum Browserfenster hervorgehen. (2P)



box1

box2

1P: Korrekte vertikale Anordnung

1P: Korrekte Breite (volle Breite)

e) Verändern Sie das CSS so, dass beide div-Elemente auf einer Zeile angezeigt werden. Fügen Sie ausserdem einen Abstand zwischen dem Textinhalt und dem Rand von 5px in alle Richtungen hinzu. Verwenden Sie möglichst wenig CSS. (3P)

```
div {  
  display: inline;  
  padding: 5px;  
}
```

Ziel minimales CSS -> Kurzschreibweise für padding, Keine überflüssige Deklarationen (3P)

4 JavaScript (20P)

4.1 Theorie (8P)

richtige Antwort: 1Punkt

falsche Antwort: -1 Punkt

leere Antwort: 0 Punkte

Sie erhalten für die Aufgabe **mindestens** 0 Punkte.

Aussage	Wahr	Falsch
JavaScript erlaubt Polymorphie		
JavaScript erlaubt Function Overloading		
Context beschreibt die Sichtbarkeit von Variablen		
Ein Objekt erlaubt Zahlen als Index z.B. <code>const obj = {}; obj[1] = 10</code>		
ESM (ECMAScript Module) sind immer im «strict mode»		
Ein Polyfill ist dafür verantwortlich, neuen JavaScript Code zu transformieren, dass der Code auch auf alten Browsern läuft.		

Aussage	Wahr	Falsch
JavaScript erlaubt Polymorphie	X	
JavaScript erlaubt Function Overloading		X
Context beschreibt die Sichtbarkeit von Variablen		X
Ein Objekt erlaubt Zahlen als Index z.B. <code>const obj = {}; obj[1] = 10</code>		X
ESM (ECMAScript Module) sind immer im «strict mode»	X	
Ein Polyfill ist dafür verantwortlich, neuen JavaScript Code zu transformieren, dass der Code auch auf alten Browsern läuft.		X

Was wird auf der Console ausgegeben?

```
let a, b, rest;
[a, b, ...rest] = [10, 20, 30, 40, 50];
console.log(rest);
```

Ausgabe:

```
// [30, 40, 50]
```

Wie nennt sich «`[a, b, ...rest]`» dieses JavaScript Feature?

```
// Destructuring assignment
```

4.2 Module (3P)

Exportieren Sie **value** als **default export** von `module.mjs`. Importieren Sie den Export im `index.mjs` und geben Sie den Wert auf der Console aus.

`module.mjs`:

```
const value = 42;
```

```
export default value 1.5
```

`index.mjs`:

```
import value from "./module.mjs 1.5P (mit console out)"  
halbe Punkte, falls komplett richtiger normaler export genutzt wurde
```

4.3 Array (6P)

Das Array bietet unterschiedliche Methoden. Wählen Sie die richtige aus und schreiben Sie den nötigen Code, um das Problem zu lösen.

1. Geben Sie alle Elemente einzeln auf der Console aus.
2. Quadrieren Sie alle Elemente.
3. Finden Sie das höchste Element im Array.

```
const arr = [3, 2, 9, 1, 6];
```

```
// 1
```

```
arr.forEach(x => console.log(x)); je 2 Punkte
```

```
arr. _____;
```

```
// 2
```

```
console.log(arr.map(x => x * x));
```

```
console.log(arr. _____)
```

```
// 3
```

```
console.log(arr.sort((a, b) => b - a)[0]);
```

```
console.log(arr. _____)
```

4.4 Functions (3P)

Was wird auf der Console ausgegeben?

a)

```
function fn1(){  
    console.log(this.text);  
}  
fn1();
```

undefined

b)

```
function fn2(...rest){  
    console.log(rest);  
}  
fn2();
```

[]

c)

```
function fn3({test}){  
    console.log(test);  
}  
fn3({test: 10});
```

10

5 DOM (6P)

Betrachten Sie folgenden HTML-Ausschnitt.

```
<button>Button 1</button>
<span></span>
<div>
  <button>Button 2</button>
  <button>Button 3</button>
</div>
```

Schreiben Sie einen Selektor, der *nur* den Button mit dem Text «Button 2» selektiert (1). Ergänzen Sie einen click-Listener (2). Wenn der Button angeklickt wird, soll im -Element der Text «clicked» erscheinen. Fügen Sie ausserdem im -Element ein Kindelement vom Typ div mit dem Text «Hello World» hinzu.

Ergänzen Sie folgenden Code. (6P)

```
<script>
// Selektieren Sie nur den Button mit dem Inhalt «Button 2» (1).
const btn =
```

```
document.querySelector("div > button:first-child"); (2P)
```

// Ergänzen Sie hier den Event-Listener für den Button (2).

```
btn.addEventListener("click", () => {
  document.querySelector("span").innerHTML = "Clicked";
  const newEl2 = document.createElement('div');
  newEl2.innerText = 'Hello World';
  document.querySelector("span").appendChild(newEl2)
}); (4P)
```

```
</script>
```

6 AJAX, REST (26P)

6.1 REST-Interfaces (4P)

a) Was ist das Merkmal eine Ressourcen-orientierte Architektur? (2P)

Es werden Ressourcen in unterschiedlichen Repräsentationen zur Verfügung gestellt. Dies ist im Gegensatz zur eher funktionalen Sicht, zum Beispiel bei RPCs.

b) Beschreiben Sie ein Merkmal von REST-Schnittstellen an einem konkreten Beispiel. (2P)

Zum Beispiel

Korrekte Verwendung der http-Verben: GET für Abfragen ohne Seiteneffekte, POST für Anlegen einer neuen Ressource

Zustandslosigkeit: In jeder Anfrage stecken alle Informationen, um diese Anfrage zu beantworten.

6.2 AJAX (8P)

a) Vervollständigen Sie den folgenden Code, der einen fetch-Request an einen REST-Endpunkt übermittelt. Der Parameter `url` ist eine Zeichenkette. Die Funktion gibt ein Promise zurück. (2P)

```
_____ getData(_____) {  
    return _____;  
}  
async function getData(url) {  
    return fetch(url);  
}
```

b) Schreiben Sie eine Funktion, die ein Promise von der Funktion `getData` erhält, die Sie in Teilaufgabe a) implementiert haben. Die Antwort des Servers enthält Daten im json-Format. Sie können entscheiden, ob Sie mit `await` oder Callbacks arbeiten möchten. (2P)

```
await fetch(SONGS_API_URL, fetchOptions);  
const insertedSong = await response.json();
```

Beschreiben Sie, wie Sie den zugehörigen Endpunkt auf dem Server implementieren würden. Es ist nicht nötig konkreten Programmcode zu schreiben.

c) Welche Response Codes können Sie verwenden? (2P)

```
200: Ok  
404: Not found  
500: Internal Server Error (1P)  
Bezug zur Anwendung nötig (keine blosse Aufzählung) (1P)
```

d) Wie können Sie eine Datei an den Client zurückgeben? (2P)

```
Reponse-Type-Header setzen  
Daten in Body schreiben
```


6.3 Darstellung von JSON-Dateien vom Server (14P)

Sie erhalten vom Server ein Array mit folgender Struktur zurück:

```
[  
  {"Band": "Test A", "SoldRecords": 123},  
  {"Band": "Test B", "SoldRecords": 456},  
  {"Band": "Test C", "SoldRecords": 1002},  
  {"Band": "Test D", "SoldRecords": 345}  
]
```

a) Es soll folgende HTML-Struktur ausgegeben werden:

```
<ul>  
  <li><strong>Test A</strong></li>  
  <li><strong>Test B</strong></li>  
  <li><strong class="high">Test C</strong></li>  
  <li><strong>Test D</strong></li>  
</ul>
```

Die Klasse high soll gesetzt werden, wenn der Wert für SoldRecords grösser ist als 1000.
Vervollständigen Sie den folgenden Codeausschnitt. (10P)

```
function show(data) {  
  let ret = "";
```

```
  var ret = "<ul>"; // 1P mit <ul>  
  dataArray.forEach((val) => { //schleife 2P  
    // Ohne class 3P  
    const liClass = val.SoldRecords > 1000 ? "high" : ""; //class 4P  
    ret += `<li class="${liClass}">${val.Band}</li>`;   
  });  
  ret += "</ul>";
```

```
// Alternative map ist auch möglich
```

```
  return ret;
```

```
}
```

b) Betrachten Sie den Namen der Funktion und des Parameters in Aufgabe a). Diskutieren Sie schriftlich anhand von Clean-Code Richtlinien, ob es sich um gute Namen handelt und wie Sie diese verbessern könnten. (4P)

show: Name ist unklar. Was tut diese Funktion tatsächlich? (1P)

Besserer Vorschlag: `renderHTML()` zeigt was die Funktion tut und welche Daten Sie zurückliefert (1P)

Data: Gleiches Problem, gleicher Verbesserungsvorschlag (1P, 1P)

7 Anhang

4.4.5 The `ol` element

Categories:[Flow content](#)If the element's children include at least one `li` element: [Palpable content](#).**Contexts in which this element can be used:**Where [flow content](#) is expected.**Content model:**Zero or more `li` and [script-supporting](#) elements.**Tag omission in text/html:**

Neither tag is omissible.

Content attributes:[Global attributes](#)`reversed` — Number the list backwards`start` — [Starting value](#) of the list`type` — Kind of list marker**Accessibility considerations:**[For authors.](#)[For implementers.](#)**DOM interface:**

```
IDL
[Exposed=Window]
interface HTMLListElement : HTMLElement {
    [HTMLConstructor] constructor();

    [CEReactions] attribute boolean reversed;
    [CEReactions] attribute long start;
    [CEReactions] attribute DOMString type;

    // also has obsolete members
};
```

The `ol` element [represents](#) a list of items, where the items have been intentionally ordered, such that changing the order would change the meaning of the document.

The items of the list are the `li` element child nodes of the `ol` element, in [tree order](#).

4.4.6 The `ul` element

Categories:[Flow content](#)If the element's children include at least one `li` element: [Palpable content](#).**Contexts in which this element can be used:**Where [flow content](#) is expected.**Content model:**Zero or more `li` and [script-supporting](#) elements.**Tag omission in text/html:**

Neither tag is omissible.

Content attributes:[Global attributes](#)**Accessibility considerations:**[For authors.](#)[For implementers.](#)**DOM interface:**

```
IDL
[Exposed=Window]
interface HTMLListElement : HTMLElement {
    [HTMLConstructor] constructor();

    // also has obsolete members
};
```

The `ul` element [represents](#) a list of items, where the order of the items is not important — that is, where changing the order would not materially change the meaning of the document.

4.4.9 The **d1** element

Categories:[Flow content](#).If the element's children include at least one name-value group: [Palpable content](#).**Contexts in which this element can be used:**Where [flow content](#) is expected.**Content model:**Either: Zero or more groups each consisting of one or more **dt** elements followed by one or more **dd** elements, optionally intermixed with [script-supporting elements](#).Or: One or more **div** elements, optionally intermixed with [script-supporting elements](#).**Tag omission in text/html:**

Neither tag is omissible.

Content attributes:[Global attributes](#)**Accessibility considerations:**[For authors](#).[For implementers](#).**DOM interface:**

```
IDL [Exposed=Window]
interface HTMLDListElement : HTMLElement {
    [HTMLConstructor] constructor();

    // also has obsolete members
};
```

The **d1** element [represents](#) an association list consisting of zero or more name-value groups (a description list). A name-value group consists of one or more names (**dt** elements, possibly as children of a **div** element child) followed by one or more values (**dd** elements, possibly as children of a **div** element child), ignoring any nodes other than **dt** and **dd** element children, and **dt** and **dd** elements that are children of **div** element children. Within a single **d1** element, there should not be more than one **dt** element for each name.

Name-value groups may be terms and definitions, metadata topics and values, questions and answers, or any other groups of name-value data.

4.4.8 The **li** element

Categories:

None.

Contexts in which this element can be used:Inside **ol** elements.Inside **ul** elements.Inside **menu** elements.**Content model:**[Flow content](#).**Tag omission in text/html:**An **li** element's [end tag](#) can be omitted if the **li** element is immediately followed by another **li** element or if there is no more content in the parent element.**Content attributes:**[Global attributes](#)If the element is not a child of an **ul** or **menu** element: **value** — [Ordinal value](#) of the list item**Accessibility considerations:**[For authors](#).[For implementers](#).**DOM interface:**

```
IDL [Exposed=Window]
interface HTMLLIElement : HTMLElement {
    [HTMLConstructor] constructor();

    [CEReactions] attribute long value;

    // also has obsolete members
};
```

The **li** element [represents](#) a list item. If its parent element is an **ol**, **ul**, or **menu** element, then the element is an item of the parent element's list, as defined for those elements. Otherwise, the list item has no defined list-related relationship to any other **li** element.

The **value** attribute, if present, must be a [valid integer](#). It is used to determine the [ordinal value](#) of the list item, when the **li**'s [list owner](#) is an **ol** element.